



UNIVERSIDADE FEDERAL DE OURO PRETO
DEPARTAMENTO DE COMPUTAÇÃO - DECOM
Professor:

Guilhermo Camara Chavéz

RELATÓRIO PRÁTICO
Disciplina: BCC221 Turma: 11

Aluno:
Carlos Henrique Nunes Souza

Morro do Cruzeiro
2024

Sumário

Sumário	2
Introdução	3
Como usar o programa:	4
Detalhes da implementação:	5
Desafios enfrentados:	6
Conceitos utilizados	7
Conclusão	8
Referências	9

Introdução

Este trabalho tem como objetivo a implementação de um sistema de gerenciamento para uma oficina mecânica. O programa foi desenvolvido na linguagem de programação C++ e foi projetado para atender as necessidades específicas de uma oficina, contemplando o cadastro de clientes, veículos, e a administração de ordens de serviço.

Como descrito nas aulas, o projeto adota uma abordagem de desenvolvimento orientada a objetos, visando a criação de um código com alta coesão e baixo acoplamento. Para isso, foram explorados os conceitos de herança e polimorfismo, garantindo que as diferentes funcionalidades fossem divididas em classes bem definidas, de forma modular e buscando ser reutilizável.

Os principais elementos do meu programa incluem os funcionários da oficina, categorizados como vendedores e mecânicos, os clientes, os veículos e os produtos e serviços oferecidos. O sistema permite que vendedores realizem o cadastro de novos clientes e veículos, gerem ordens de serviço, e acompanhem o processo de aprovação de orçamentos. Os mecânicos podem acessar as ordens de serviço, detalhar os serviços executados e registrar as peças utilizadas. Adicionalmente, o sistema inclui um administrador responsável pela gestão dos dados dos funcionários.

Através deste sistema, o programa busca otimizar o fluxo de trabalho na oficina, garantindo maior controle sobre as operações diárias, desde a recepção do cliente até a execução e fechamento dos serviços prestados. Tudo isso usando conceitos de programação orientada a objetos.

A implementação do trabalho foi feita usando a **IDE Visual Studio Code da Microsoft**.
O Sistema operacional utilizado foi o **Windows 11 Home Single Language Versão 23H2**.

Como usar o programa:

Para compilar e executar o programa, existe um arquivo Makefile na raiz do projeto como descrito na imagem:

Usando o sistema operacional Windows, precisa-se instalar uma ferramenta chamada MinGW ou CMake. Para o MinGW, basta garantir que os pacotes mingw32-base e mingw32-gcc-g++ estão selecionados durante a instalação do mesmo. Após instalado, basta executar o comando mingw32-make, após isso as instruções dentro do arquivo Makefile serão executadas e o programa será compilado.

Para o Linux, basta digitar o comando make dentro da pasta src pois o make já é nativo do sistema.

Para executar o programa acesse a pasta bin e execute o **main**

Depois de executado, o programa irá exibir a seguinte tela:

```
----- MENU PRINCIPAL -----  
1. Login  
2. Sair do sistema  
Escolha uma opção: 1  
A qualquer momento digite 0 para voltar
```

onde o usuário terá acesso a um menu onde ele pode fazer o login ou sair do programa digitando a opção, no caso **1 ou 2**.

ao selecionar **1** para **login** aparecerá esta tela:

```
----- LOGIN -----  
Login: vendedor  
Senha: vendedor
```

onde o usuário poderá logar como **Administrador, Mecânico ou Vendedor**

para logar como administrador :

login: admin

senha: admin

ao logar como administrador aparecerá a seguinte tela:

```
Usuario autenticado com sucesso.  
Bem vindo lucas  
Seu tipo de usuario é: Admin  
----- MENU ADMINISTRADOR -----  
Para voltar digite 0  
1. Editar dados de Vendedores  
2. Editar dados de Mecânicos  
3. Adicionar Vendedor  
4. Adicionar Mecânico  
5. Remover Vendedor  
6. Remover Mecânico  
Escolha uma opção: █
```

onde o usuário **terá 6 opções (1,2,3,4,5,6)**:

Na primeira e segunda opção, o administrador poderá editar os dados dos vendedores e dos mecânicos, inclusive a senha e login deles

Na terceira opção e quarta opção o administrador poderá adicionar um novo mecânico ou vendedor passando todas as informações de cada inclusive o login deles

Na quinta opção e sexta opção ele poderá escolher um dos vendedores ou mecânicos para remover do sistema.

Para logar como vendedor: (obs: no programa já é iniciado 3 vendedores padrões para facilitar)

login: vendedor

login: vendedor

ao logar como vendedor aparecerá a seguinte tela:

```
Bem vindo joao  
Seu tipo de usuario é: Vendedor  
----- MENU VENDEDOR -----  
1. Cadastrar cliente e veículo  
2. Gerar ordem de serviço  
3. Visualizar ordens de serviço de orçamento pendentes de aprovação do cliente e marcar como aprovadas  
4. Visualizar ordens de serviço executadas e realizar o fechamento
```

onde o usuário **terá 4 opções (1,2,3,4)**:

Caso queira fazer um preenchimento automático de dados para clientes existe um função comentada no Case 1 do menu mecanico

“sistema->preenccherClientesOrdem();”

Na opção 1 o vendedor cadastra o cliente com todos os seus dados e os veículos de cliente com todos seus dados, podendo ser mais de 1 veículo.

Na opção 2 o vendedor gera uma ordem de serviço para algum dos veículos do cliente, passando a data, a descrição do que precisa ser feito, e o valor que pode ser passado como 0 para fazer um orçamento com mecânico ou pode passar o valor padrão do serviço solicitado pelo cliente marcando também como pendente.

Na opção 3 o vendedor visualiza as ordens de serviço pendentes de aprovação e seleciona alguma através do número de identificação da ordem no caso 1, 2,3 etc. Caso os mecânicos tenham já feito o orçamento o valor do serviço estará atualizado com os serviços e o vendedor perguntará o cliente se ele quer continuar se sim marca como aprovada se não ele cancela o serviço.

Na opção 4 o vendedor visualiza as ordens de serviços já executadas pelo mecânico e realiza o fechamento selecionando o número da ordem de serviço pois ela já tem o valor atualizado do serviços executados.

Para logar como vendedor: (obs:no programa já é iniciado 3 mecânicos padrões para facilitar)

login: mecanico

login: mecanico

ao logar como mecânico aparecerá a seguinte tela:

```
Usuario autenticado com sucesso!  
Bem vindo carlos  
Seu tipo de usuario é: Mecanico  
----- MENU MECÂNICO -----  
1. Visualizar ordens de serviço abertas.  
2. Visualizar ordens de serviço pendentes para atualizar o valor total do servico e pecas.  
3. Cadastrar serviços executados e peças utilizadas.
```

onde o usuário **terá 3 opções (1,2,3):**

Na opção 1 o mecânico verá todas as ordens de serviços abertas (as Pendentes e Aprovadas) com detalhes sobre cada uma delas.

Na opção 2 o mecânico verá as ordens de serviços pendentes, que estão esperando o orçamento de peças e serviços, nesta tela ele terá a opção de escolher uma ordem e adicionar cada serviço e cada valor de serviço e cada peça que será usada no serviço e cada valor de peça, no final terá a soma do serviço completo daquela ordem onde o vendedor irá acessar e vai passar pro cliente.

Na opção 3 o mecânico verá os serviços aprovados para manutenção e ele poderá selecionar um deles para executar, passando a peças e serviços e valores, caso a ordem esteja com o valor do orçamento e os serviços executados é o mesmo ele poderá passar o valor de 0 para cada peça e serviço pois já está feita quando foi feito o orçamento.

Desafios enfrentados:

Durante o desenvolvimento do sistema de gerenciamento para a oficina mecânica, vários desafios surgiram:

1. **Integração de Funcionalidades:** Um dos maiores desafios foi garantir que todas as funcionalidades do sistema trabalhassem juntas de forma harmoniosa. Com diferentes tipos de usuários, como administradores, vendedores e mecânicos, era crucial assegurar que as interações entre essas partes fossem coerentes e que o fluxo de dados fosse bem gerenciado.
2. **Manutenção de Dados e Coerência:** Manter a integridade dos dados ao realizar operações como adicionar, remover e atualizar informações foi um desafio constante. O sistema precisava garantir que todas as ações de criar, ler, atualizar e deletar fossem feitas corretamente, sem causar inconsistências nos dados.
3. **Interface do Usuário:** Desenvolver uma interface que fosse fácil de usar e intuitiva para todos os tipos de usuários também apresentou dificuldades. Era essencial que as telas e menus fossem claros e funcionais, o que demandou uma série de testes e ajustes.
4. **Gerenciamento de Memória e Recursos:** Gerenciar a memória e os recursos de forma eficiente foi um desafio importante. O uso adequado de ponteiros e a liberação correta de memória foram fundamentais para evitar vazamentos e garantir que o sistema funcionasse sem problemas de desempenho.

Detalhes da implementação:

A implementação do sistema foi baseada em uma arquitetura orientada a objetos. Foram utilizadas diversas classes para representar os diferentes atores e entidades do sistema, como funcionários, clientes, veículos e ordens de serviço.

Estrutura de Classes

1. **Admin:** A classe Admin herda da classe base **Funcionario**. Esta classe foi projetada para representar o administrador do sistema, permitindo-lhe gerenciar vendedores e mecânicos. A classe implementa um construtor que inicializa os atributos do administrador, além de métodos para retornar o tipo de funcionário e gerenciar o sistema.
2. **Clientes:** A classe Clientes representa os clientes da oficina. Ela contém informações detalhadas sobre o cliente, como nome, CPF, telefone, endereço e uma lista de veículos associados ao cliente. Métodos foram implementados para adicionar e acessar os veículos do cliente, além de imprimir os dados do cliente.
3. **Funcionario:** A classe Funcionario é uma classe abstrata que serve como base para os tipos específicos de funcionários, como **Mecanicos e Vendedores**. Ela armazena informações comuns a todos os funcionários, como login, senha, nome e CPF, além de métodos para autenticação e impressão dos dados.
4. **Mecanicos:** A classe **Mecanicos** herda de **Funcionario** e representa os mecânicos da oficina. Ela permite que os mecânicos acessem as ordens de serviço associadas e adicionem os serviços executados e peças utilizadas.
5. **OrdemServico:** Esta classe modela as ordens de serviço no sistema. Ela contém informações como a data de criação, descrição, status, valor, veículo e cliente associados. Métodos foram implementados para adicionar peças e serviços à ordem, bem como para modificar e acessar seus atributos.
6. **SistemaLogin:** A classe **SistemaLogin** gerencia a autenticação e o armazenamento de usuários (funcionários) e ordens de serviço. Ela permite adicionar, remover e autenticar usuários, além de acessar as listas de clientes e ordens de serviço.
7. **Veiculos:** A classe Veículos representa os veículos dos clientes, armazenando informações como placa, marca, modelo, cor, ano, quilometragem e tipo de combustível. Métodos para acessar e modificar esses atributos, bem como imprimir as informações do veículo, foram implementados.
8. **Vendedores:** A classe Vendedores, também herdeira do Funcionario, representa os vendedores da oficina. Ela permite adicionar clientes e ordens de serviço ao sistema, além de acessar e gerenciar esses dados.

Detalhes das Classes

Funcionario:

A classe **Funcionario** tem como atributos protected login e senha, com isso as classes que herdam de **Funcionario** podem acessar estes atributos. Os atributos que dizem respeito a informações pessoais sobre o funcionário são armazenadas como private para evitar que as mesmas sejam alteradas resultando em inconsistências. Foi implementado também um método **autenticar**, definido como virtual para que as classes derivadas possam implementá-lo. Seguindo a linha de métodos virtuais, foi criado também um método virtual puro chamado **getTipo()**, este método pega o tipo do funcionario atual e o retorna. A nomenclatura **const = 0** faz com que cada classe que herdar de **Funcionario** tenha que implementar esse método

```
C/C++
class Funcionario {
protected:
    string login;
    string senha;
private:
    int id;
    string nome;
    string cpf;
    string telefone;
    string email;
public:
    Funcionario(
        const string& login,
        const string& senha,
        int id,
        string nome,
        string cpf,
        string telefone,
        string email
    );

    ~Funcionario();

    virtual bool autenticar(string& login, string& senha);
    virtual string getTipo() const = 0;
};
```

Cliente

A classe Cliente possui alguns atributos privados, dentre eles, destaca-se um vetor chamado **veiculos** que armazena **ponteiros** para objetos **Veiculo**, que por sua vez representam os veículos de um determinado cliente. Dentre os métodos encontra-se o **getVeiculos()** que retorna os veículos do cliente em questão.

```
C/C++
class Clientes {
private:
    int id;
    string nome;
    string cpf;
    string telefone;
    string email;
    string rua;
    string bairro;
    string cidade;
    string estado;
    string cep;
    string numero;
    vector<Veiculos*> veiculos;
public:
    Clientes(
        int id,
        const string& nome,
        const string& cpf,
        const string& telefone,
        const string& email,
        const string& rua,
        const string& bairro,
        const string& cidade,
        const string& estado,
        const string& cep,
        const string& numero
    );

    ~Clientes();

    //Setters: métodos para definir os atributos do cliente
    //Getters: métodos para acessar os atributos do cliente

    vector<Veiculos*> getVeiculos() const
    int getQuantosVeiculos();
};
```

Admin

A classe **Admin** herda atributos e métodos da classe **Funcionário**, dentre eles destaca-se o método **getTipo()** definido anteriormente como virtual puro e implementado corretamente nesta classe. Fora isso, a **Admin** possui um **construtor** que popula os atributos herdados de **Funcionário**.

```
C/C++
class Admin : public Funcionario {

public:
    Admin(
        const string& login,
        const string& senha,
        int id,
        string nome,
        string cpf,
        string telefone,
        string email
    );

    ~Admin();

    string getTipo() const override;
};
```

Mecanico:

A Classe **Mecanico** também herda de **Funcionario** e popula alguns atributos da mesma. Dentro dela também temos o método **getTipo()** e alguns métodos auxiliares para obter informações que um mecânico necessita, como por exemplo a ordem de serviço.

C/C++

```
class Mecanicos : public Funcionario {
public:
    Mecanicos(
        const string& login,
        const string& senha,
        int id,
        string nome,
        string cpf,
        string telefone,
        string email
    );

    ~Mecanicos();

    string getTipo() const override;
    vector<OrdemServico*> getOrdemServico(SistemaLogin& sistema);
    int getQuantosOrdemServico(SistemaLogin& sistema);
};
```

Vendedores

A classe **Vendedores** também entra no escopo das que herdam de **Funcionario**. Seu construtor popula os atributos da **superclasse**, e são implementados também alguns métodos para o Vendedor gerenciar o sistema como por exemplo o **addCliente** e o **removeCliente** que fazem a gestão da inclusão/exclusão de clientes na mecânica.

C/C++

```
class Vendedores : public Funcionario {

public:
    Vendedores(
        const string& login,
        const string& senha,
        int id,
        string nome,
        string cpf,
        string telefone,
        string email
    );

    ~Vendedores();
    string getTipo() const override;

    void addCliente(Clientes* cliente, SistemaLogin& sistema);
    void addOrdemServico(OrdemServico* ordem, SistemaLogin& sistema);

    vector<Clientes*> getClientes(SistemaLogin& sistema);
    vector<OrdemServico*> getOrdemServico(SistemaLogin& sistema);

    int getQuantosClientes(SistemaLogin& sistema);
    int getQuantosOrdemServico( SistemaLogin& sistema);

    void removeCliente(int id, SistemaLogin& sistema);
    void removeOrdemServico(int id, SistemaLogin& sistema);

};
```

Veiculos

A classe **Veiculos** gerencia o cadastro de veículos dos clientes, nela basicamente temos os métodos que manipulam os atributos dos diferentes carros.

C/C++

```
class Veiculos {
private:
    int id;
    string placa;
    string marca;
    string modelo;
    string cor;
    string ano;
    int km;
    string combustivel;
public:
    Veiculos(
        int id,
        string placa,
        string marca,
        string modelo,
        string cor,
        string ano,
        int km,
        string combustivel
    );
    ~Veiculos();

    // Métodos setters para alterar os atributos do veículo
    // Métodos getters para acessar os atributos do veículo
};
```

OrdemServiço

A Classe **OrdemServiço** gerencia a ordem de serviço performada em um veículo. Essa classe armazena um vetor de peças e serviços performados em um veículo e possuem **getters** e **setters** para a gestão dos atributos da mesma.

```
C/C++
class OrdemServico {
private:
    int id;
    string data;
    string descricao;
    string status;
    string valor;
    string veiculo;
    string cliente;
    vector<string> pecas;
    vector<string> servicos;

public:
    OrdemServico(
        int id,
        string data,
        string descricao,
        string status,
        string valor,
        string veiculo,
        string cliente
    );

    ~OrdemServico();

    //Getters para acessar os atributos da ordem de serviço
    //Setters para modificar os atributos da ordem de serviço
    void addPeca(string peca);
    void addServico(string servico);
};
```

SistemaLogin

A classe **SistemaLogin** gerencia o login e a autenticação dos usuários no sistema, além de armazenar informações críticas, como listas de **Funcionarios**, **Clientes**, e **OrdemServico**. Os vetores são utilizados para manter essas coleções de objetos, facilitando o gerenciamento de múltiplos usuários e dados associados. A classe também concede acesso especial a determinadas funcionalidades para as classes **Vendedores e Mecanicos** através do uso de **friend class**, permitindo que essas classes acessem diretamente os dados encapsulados. Essa abordagem assegura que as classes derivadas possam interagir de maneira controlada com as informações armazenadas em **SistemaLogin**.

C/C++

```
class SistemaLogin {
private:
    vector<Funcionario*> usuarios;
    vector<Clientes*> clientes;
    vector<OrdemServico*> servicos;
    friend class Vendedores;
    friend class Mecanicos;
public:
    SistemaLogin();
    ~SistemaLogin();

    void addUsuario(Funcionario* usuario);
    Funcionario* autenticar(string& login, string& senha) const;

    //Getters e Setters para os atributos da classe

    //Remove um usuário do sistema pelo id
    void removeUsuario(int id);
    void preencherClientesOrdem();
};
```


Telas

A classe **Telas** gerencia as interfaces de menu do sistema, oferecendo opções específicas para diferentes tipos de usuários. Ela usa um atributo **SistemaLogin*** sistema para acessar dados e funcionalidades do sistema. A classe inclui métodos para exibir menus para **mecânicos**, **vendedores**, **administradores** e para **autenticação de usuários**. Além disso, o método **lerString** captura entradas dos usuários para garantir uma interação eficiente com o sistema.

```
C/C++
class Telas {
private:
    SistemaLogin *sistema = new SistemaLogin();

public:
    Telas();
    ~Telas();
    void menuPrincipal();
    void menuMecanicos(Mecanicos* mecanico, SistemaLogin* sistema);
    void menuVendedores(Vendedores* vendedor, SistemaLogin* sistema);
    void menuAdmin(SistemaLogin* sistema);
    void menuLogin();
    string lerString();
};
```

Criei um modelo UML utilizando o programa Astah UML, mas o arquivo resultante acabou ficando muito extenso. Para melhor ilustrar as relações entre as classes e os métodos de cada uma, incluí o arquivo UML.pdf na pasta “docs/uml” do meu projeto.

Conceitos utilizados

No desenvolvimento do sistema de gerenciamento para a oficina mecânica, foram aplicados vários conceitos essenciais de programação orientada a objetos e design de software:

Orientação a Objetos: A estrutura do sistema foi organizada com base nos princípios da orientação a objetos. Isso permitiu criar uma modelagem clara e modular, utilizando classes como **Funcionario**, **Clientes**, **Veiculos** e **OrdemServico** para encapsular dados e comportamentos, o que facilitou a organização e a manutenção do código.

Herança e Polimorfismo: A herança foi utilizada para estabelecer uma hierarquia de classes, com **Funcionario** como base e suas subclasses **Admin**, **Mecanicos** e **Vendedores** estendendo e especializando o comportamento. O polimorfismo, por meio de métodos virtuais, permitiu que as subclasses implementassem comportamentos específicos, como o método **getTipo()**.

Encapsulamento: Para proteger os dados internos das classes e garantir que as interações com esses dados sejam controladas, os atributos e métodos foram cuidadosamente organizados. Informações sensíveis, como login e senha, foram mantidas privadas e acessíveis apenas através de métodos públicos.

Abstração: A abstração ajudou a criar representações simplificadas dos componentes do sistema, como veículos, clientes e ordens de serviço. Isso permitiu focar nas funcionalidades essenciais sem se perder em detalhes de implementação, tornando a interface do usuário mais clara e as responsabilidades das classes mais definidas.

Design Modular: O projeto foi desenvolvido com uma abordagem modular, dividindo o sistema em partes distintas com responsabilidades bem definidas. Isso facilitou a reutilização do código e tornou a manutenção e a expansão do sistema mais simples.

Gerenciamento de Memória: A gestão eficiente da memória foi crucial para o desempenho do sistema. O uso de ponteiros e a correta liberação de memória foram essenciais para evitar vazamentos e garantir que o sistema funcionasse de maneira eficaz.

Conclusão

Neste projeto, desenvolvi um sistema de gerenciamento para uma oficina mecânica em C++. A principal tarefa foi criar uma aplicação que gerencie o cadastro de clientes e veículos, e facilite o processo de geração e execução de ordens de serviço.

Ao longo do desenvolvimento, apliquei conceitos de orientação a objetos, como herança e polimorfismo, para construir um código bem estruturado e modular. Aprendi a organizar melhor as funcionalidades em diferentes classes e a manter uma comunicação clara entre elas.

O sistema permite que vendedores e mecânicos desempenhem suas funções de forma mais eficiente e que o administrador possa gerenciar os dados dos funcionários. Em resumo, o projeto proporcionou uma oportunidade prática para aplicar e consolidar conhecimentos em programação e design de software, resultando em uma ferramenta funcional para a oficina atingindo todos os requisitos solicitados no enunciado do trabalho.

Referências

Microsoft Documentation. (2024). *Visual Studio Code Documentation*.

- Disponível em: <https://code.visualstudio.com/docs>
- Documentação oficial da IDE utilizada para o desenvolvimento do sistema.

GNU Make Manual. (2024). *Make - A GNU Manual*.

- Disponível em: <https://www.gnu.org/software/make/manual/make.html>
- Guia sobre o uso do **make**, uma ferramenta essencial para a construção e compilação do projeto.

Astah UML. (2024). *Astah UML Documentation*.

- Disponível em: <https://astah.net/astah-uml>
- Documentação sobre o uso do Astah UML para modelagem e visualização das relações entre as classes.